

***Curso de Ciência de Dados, Machine Learning,
Deep Learning e IA generativa com uso de LLMs***

Disciplina: Visão Computacional e IA Generativa
Professor(a): João Macedo

Agenda - Bloco 1

Fundamentos de Edge Computing

1 - Definições essenciais

- O que é Edge Computing e por que importa
- Sistemas embarcados, IoT e dispositivos de borda
- Edge AI vs Cloud AI: quando usar cada um

2 - Por que precisamos de Edge AI?

- O modelo BLERP: Bandwidth, Latency, Economics, Reliability, Privacy
- Casos de uso reais em indústria, saúde e campo

3 - Pipeline e formatos de deploy

- Visão geral do fluxo: treinamento → otimização → inferência
- Formatos de modelos: ONNX, TensorRT, TFLite, CoreML, ncnn
- Trade-offs: acurácia vs velocidade vs tamanho vs consumo

O que é Edge Computing?

Fundamentos de Edge Computing

Computação na borda da rede

- Edge Computing significa processar dados perto de onde são gerados
- Oposto da abordagem Cloud: não há round-trip para um servidor remoto
- O dado é analisado no próprio dispositivo ou em um servidor local próximo

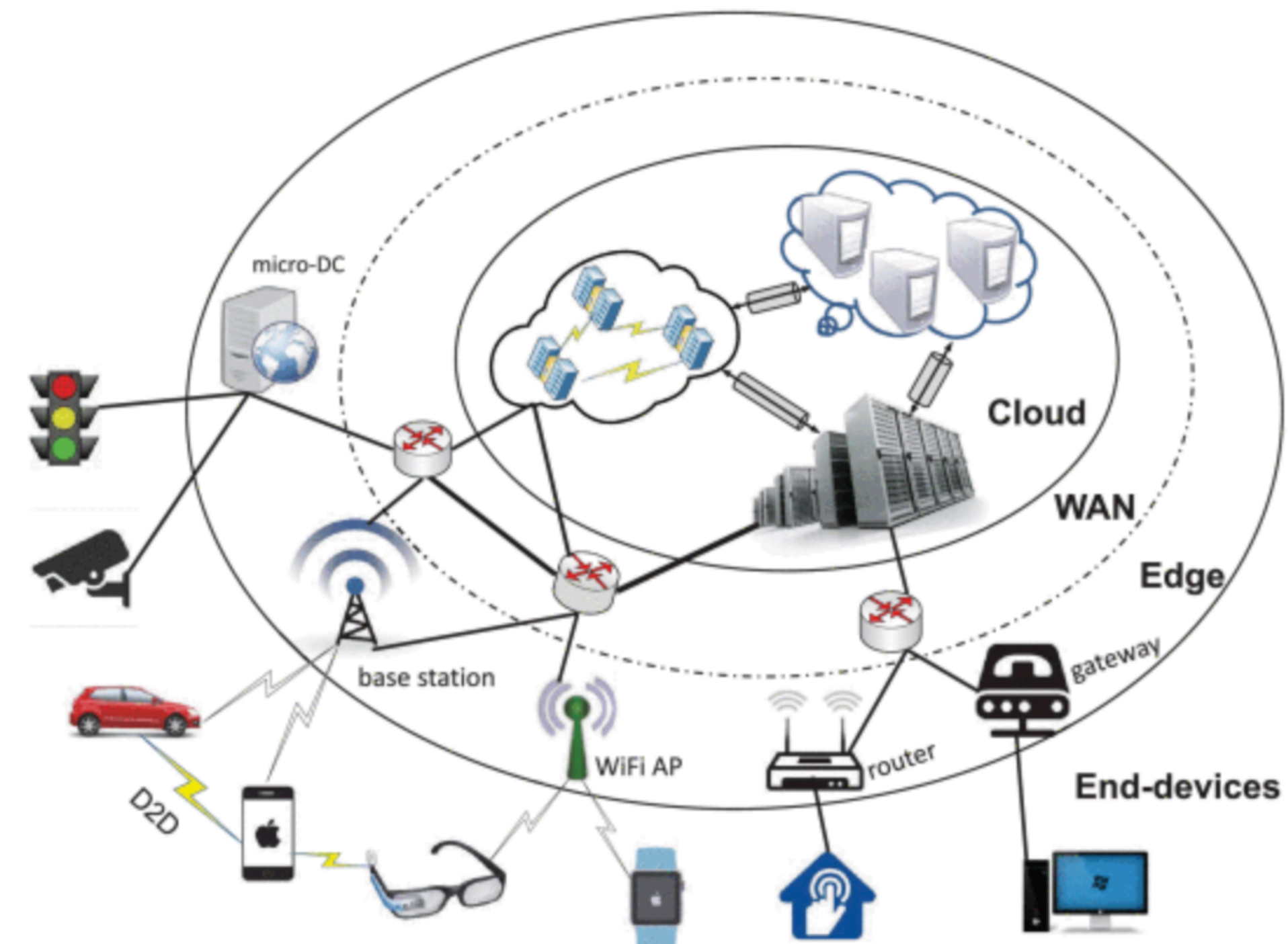


Imagem: Zhou et al. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing, IEEE, 2019

O que é Edge Computing?

Fundamentos de Edge Computing

Origem: da centralização a distribuição

- Anos 1960: mainframes centralizados (toda a computação em um lugar)
- Anos 1980: computadores pessoais - processamento começou a se distribuir
- Anos 2000: cloud computing - volta ao centralizado, mas via internet
- Hoje: IoT e Edge - o processamento se move de volta para a borda

O que é Edge Computing?

Fundamentos de Edge Computing

Definição de dispositivo de borda (edge device)

- Qualquer computador embutido na borda da rede: sensores, cameras, drones, carros
- Em 2021 havia mais de 12 bilhões de dispositivos IoT conectados
- Smartphones, tablets e PCs também são dispositivos de borda

Sistemas Embarcados e IoT

Fundamentos de Edge Computing

Sistemas embarcados

- Computadores dedicados a uma tarefa especifica dentro de um dispositivo
- Exemplos: controle de motor de carro, fone bluetooth, console de jogos
- Restricoes tipicas: memoria limitada, bateria, clock reduzido, sem tela
- Programar sistemas embarcados e navegar restricoes - arte da otimizacao

Sistemas Embarcados e IoT

Fundamentos de Edge Computing

Internet das Coisas (IoT)

- Rede de objetos físicos com processadores e conectividade de rede
- Sensores industriais, cameras de seguranca, rastreadores, eletrodomesticos
- Problema classico: os dispositivos geram mais dados do que conseguem transmitir
- Solucao: processar na borda e enviar apenas o que for relevante

Sistemas Embarcados e IoT

Fundamentos de Edge Computing

Edge AI = Inteligência Artificial em dispositivos de borda

- Execução de modelos de ML diretamente no dispositivo, sem necessidade de nuvem
- Embedded ML: ML em sistemas embarcados
- TinyML: ML em microcontroladores

Por que Edge AI? O modelo BLERP

Fundamentos de Edge Computing

BLERP - Os 5 benefícios fundamentais do Edge AI

B - Bandwidth (Largura de Banda)

- Sensores geram volumes enormes de dados de baixo valor informacional
- Transmitir tudo consome energia e largura de banda inexistentes
- Com Edge AI: só o resultado (predição) precisa ser transmitido

L - Latency (Latência)

- Round-trip para a nuvem pode levar dezenas a centenas de milissegundos
- Aplicações críticas (veículos, drones, robotica) exigem resposta em < 10 ms
- Edge AI elimina o round-trip: decisão tomada localmente, de imediato

E - Economics | R - Reliability | P - Privacy

- Economics: sem custos de conectividade e infra de nuvem por requisicao
- Reliability: funciona sem internet - robusto a falhas de conexao
- Privacy: dados sensíveis nunca saem do dispositivo

Casos de Uso Reais de Edge AI

Fundamentos de Edge Computing

Segurança Pública

- Videomonitoramento inteligente em tempo real
 - > Comportamentos suspeitos
 - > Reconhecimento Facial
 - > Placas de veículos
 - > Drones para monitoramento e busca
- Triagem de evidências forenses

Saúde

- Detecção de queda em idosos sem enviar vídeo para a nuvem

Casos de Uso Reais de Edge AI

Fundamentos de Edge Computing

Industria e Infraestrutura

- Controle de qualidade em linha de produção com visão computacional
- Monitoramento de vibração em máquinas industriais

Autonomia e Mobilidade

- Veículos autônomos: todo o processamento perceptivo ocorre no carro
- Drones: detecção de obstáculos e objetos em tempo real sem conexão

Meio Ambiente

- Monitoramento de plantações com drones com visão computacional

O Continuum Cloud-to-Edge

Fundamentos de Edge Computing

Não é binário: há vários níveis de distribuição

Níveis de inteligência distribuída (Zhou et al., IEEE 2019)

- Nível 0 - Cloud Intelligence: treinamento e inferência 100% na nuvem
- Nível 1 - Cloud training + cloud/edge inference: modelo na nuvem, inferência híbrida
- **Nível 2 - Cloud training + edge inference: modelo treinado na nuvem, roda na borda**
- **Nível 3 - Cloud training + on-device inference: modelo no próprio dispositivo final**
- Nível 4 - Cloud-edge co-training: treinamento distribuído entre nuvem e borda
- Nível 5 - Edge training and inference: tudo na borda (ex: federated learning)

Imagem: Zhou et al. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing, IEEE, 2019

O Continuum Cloud-to-Edge

Fundamentos de Edge Computing

Níveis de inteligência distribuída (Zhou et al., IEEE 2019)

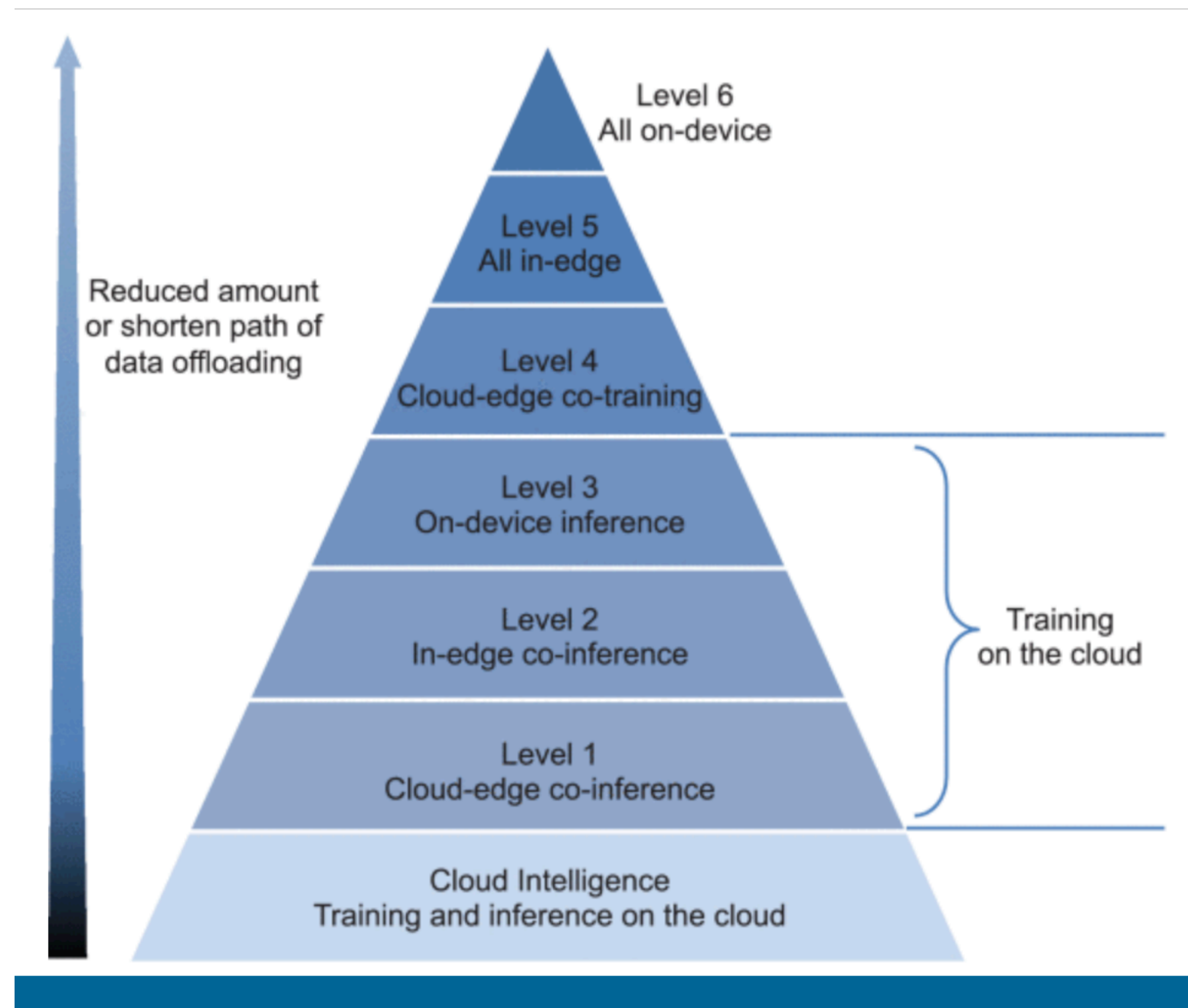


Imagem: Zhou et al. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing, IEEE, 2019

Hardware para Edge AI

Fundamentos de Edge Computing

Espectro de dispositivos - do mais ao menos restrito

- Microcontroladores (MCU): ARM Cortex-M, ESP32 - sem SO, KB de RAM (TinyML)
- Single-Board Computers: Raspberry Pi 4/5, Orange Pi - Linux, GB de RAM
- Edge GPUs: NVIDIA Jetson Nano/Orin, Google Coral - NPU/GPU dedicado
- Computadores x86 sem GPU discreta: inferência em CPU (nosso foco principal)
- Servidores locais com GPU pequena: NVIDIA RTX 3060/4060, T4

Aceleradores dedicados para inferência

- NVIDIA Jetson: GPU Pascal/Ampere compacta, suporte nativo a TensorRT
- Intel Neural Compute Stick / OpenVINO: aceleração em CPU Intel e Movidius
- Google Coral TPU: aceleração em dispositivos com TFLite quantizado INT8
- Qualcomm AI Engine: presente em smartphones Android modernos

Hardware para Edge AI

Fundamentos de Edge Computing

Cenário típico para Segurança Pública

- Computador sem GPU compatível com CUDA ou servidor com GPU pequena
- Servidores locais com GPUs médias
- Possibilidade de uso de Drones?

Pipeline de Deploy - Visão Geral

Fundamentos de Edge Computing

Fluxo completo: do modelo treinado até a inferência na borda

1 - Treinamento (servidor com GPU potente)

- Framework: PyTorch | Bibliotecas: timm, transformers, ultralytics
- Resultado: arquivo .pt ou .pth com pesos do modelo

2 - Otimização e Conversão

- Quantização: FP32 → FP16 ou INT8 (reduz tamanho e aumenta velocidade)
- Pruning: remove conexões redundantes sem perda significativa de acurácia
- Exportação: PyTorch → ONNX → TensorRT/ TFLite



Pipeline de Deploy - Visão Geral

Fundamentos de Edge Computing

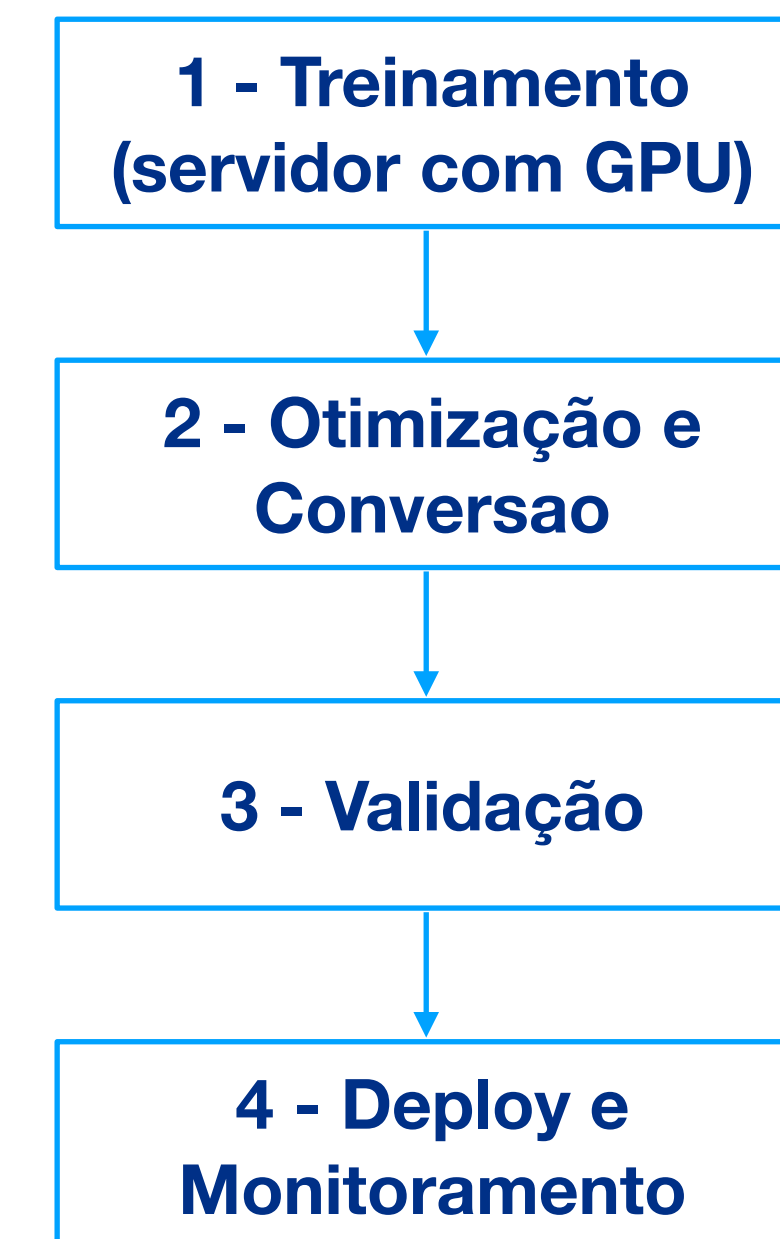
Fluxo completo: do modelo treinado até a inferência na borda

3 - Validação

- Comparar acurácia do modelo original vs modelo otimizado
- Benchmarking de latência e throughput no hardware-alvo
- Testes de robustez: iluminação variada, oclusão, diferentes ângulos

4 - Deploy e Monitoramento

- Empacotamento do runtime de inferência com o modelo
- Monitoramento de performance em produção (drift, degradação)



Formatos de Modelos para Inferência

Fundamentos de Edge Computing

Por que existem tantos formatos?

- Cada hardware e framework tem representação interna diferente
- Formatos intermediários buscam portabilidade e otimização

Formatos de Modelos para Inferência

Fundamentos de Edge Computing

Principais formatos e seus usos

- **ONNX**: formato aberto, portátil, suportado pela maioria dos runtimes
- **TensorRT** (NVIDIA): engine binário altamente otimizado para GPUs NVIDIA
- **TFLite**: Google, para Android, iOS e microcontroladores
- **ncnn**: framework leve para dispositivos mobile e ARM sem dependências
- **OpenVINO IR**: Intel, otimizado para CPU Intel, VPU e GPU Intel integrada

Hierarquia típica de conversão com PyTorch

- PyTorch (.pt) → ONNX (.onnx) → TensorRT (.engine) ou OpenVINO (IR)
- Ultralytics YOLOv8: exporta diretamente para todos os formatos

Trade-offs em Modelos de Edge

Fundamentos de Edge Computing

O triângulo impossível do deploy em Edge

- Você não pode maximizar acurácia, velocidade e tamanho ao mesmo tempo
- É necessário escolher quais dimensões priorizar para o caso de uso

Acurácia vs Tamanho

- Modelos maiores tendem a ser mais acurados, mas consomem mais memória
- Quantização INT8 pode reduzir tamanho em 4x com queda de $< 1\%$ em acurácia
- Pruning pode reduzir 50-90% dos pesos com impacto moderado em acurácia

Trade-offs em Modelos de Edge

Fundamentos de Edge Computing

Acurácia vs Velocidade (latência)

- **FP32**: máxima acurácia, mais lento | **FP16**: equilíbrio | **INT8**: mais rápido
- **TensorRT**: kernels fusionados e calibrados para a GPU alvo = ganhos 2-10x
- Modelos enxutos (**MobileNet**, **EfficientNet-lite**, **YOLOv8n**): projetados para Edge

Tamanho vs Memória em tempo de execução

- Peso do modelo em disco != memória RAM necessária durante inferência
- Ativações intermediárias podem ser maiores que o próprio modelo
- Batch size = 1 é obrigatório em alguns deploys em Edge

Agenda - Bloco 2

ONNX, TensorRT e Deploy Prático

1 - ONNX

- Estrutura do grafo ONNX e operadores suportados
- Exportando modelos PyTorch para ONNX com `torch.onnx.export`
- Inferência com ONNX Runtime (CPU e GPU)

2 - Quantização de modelos

- FP32, FP16, INT8: diferenças práticas e impacto em modelos de visão
- Exemplo: modelo timm (ResNet50, EfficientNet) quantizado para FP16
- Post-Training Quantization vs Quantization-Aware Training

Agenda - Bloco 2

ONNX, TensorRT e Deploy Prático

3 - TensorRT

- Arquitetura do TensorRT: parser, otimizador, engine
- Construindo engines TensorRT a partir de ONNX
- Precisão FP16 e INT8 com calibracao
- Comparativo de performance: PyTorch vs ONNX vs TensorRT

4 - Deploy sem GPU: alternativas e exemplos

O que é ONNX?

ONNX, TensorRT e Deploy Prático

Open Neural Network Exchange

- Formato aberto criado pela Microsoft e Facebook (Meta) em 2017
- Representa modelos de ML como grafos computacionais interoperáveis
- Suportado por PyTorch, TensorFlow, Keras, scikit-learn e dezenas de outros
- Permite exportar de um framework e executar em outro sem reescrever código

O que é ONNX?

ONNX, TensorRT e Deploy Prático

Por que ONNX é o ponto de partida ideal para deploy?

- Formato intermediário universal: exporta uma vez, roda em qualquer lugar
- ONNX Runtime (ORT): motor de inferência otimizado, multiplataforma
- Suporte a backends: CPU, CUDA, TensorRT, OpenVINO, DirectML, CoreML
- Ampla compatibilidade com hardware: x86, ARM, GPU NVIDIA, Intel, Apple

O que é ONNX?

ONNX, TensorRT e Deploy Prático

Componentes do ecossistema ONNX

- ONNX Model Zoo: modelos pré-treinados prontos para uso em ONNX
- ONNX Runtime: inferência eficiente (Microsoft)
- Netron: visualizador de grafos ONNX - útil para debug
- onnxsim / onnx-optimizer: simplificam e otimizam o grafo

Estrutura do Grafo ONNX

ONNX, TensorRT e Deploy Prático

ONNX representa um modelo como um grafo direcionado aciclico (DAG)

- Nós (nodes): operações matemáticas (Conv, ReLU, BatchNorm, Gemm, etc.)
- Arestas: tensores que fluem entre os nós - entradas e saídas
- Inicializadores (initializers): pesos e biases do modelo
- Inputs/Outputs: entradas e saídas do modelo com shapes e tipos definidos

Estrutura do Grafo ONNX

ONNX, TensorRT e Deploy Prático

Versoes e opsets ONNX

- ONNX usa o conceito de opset version (versão do conjunto de operadores)
- opset 17+ recomendado para modelos modernos
- Nem todos os operadores estão disponíveis em todos os backends
- Ao exportar: usar opset compatível com o backend alvo

Estrutura do Grafo ONNX

ONNX, TensorRT e Deploy Prático

Verificando o grafo exportado

- `onnx.checker.check_model(model)`: valida estrutura e tipos
- Netron (`netron.app`): visualização interativa do grafo no browser
- `onnxruntime.InferenceSession`: valida que o modelo executa corretamente

Exportando PyTorch para ONNX

ONNX, TensorRT e Deploy Prático

Dois metodos principais no PyTorch moderno

- `torch.onnx.export` (legado, estável): baseado em TorchScript
- `torch.onnx.dynamo_export` (PyTorch 2.x+): baseado em `torch.export` - mais flexível

Parametros criticos do `torch.onnx.export`

- `input_names` / `output_names`: nomear entradas e saídas para uso no runtime
- `dynamic_axes`: definir quais dimensões são dinamicas (batch, height, width)
- `opset_version`: versão do opset ONNX - usar 17 como padrão atual
- `do_constant_folding=True`: dobra constantes no grafo (otimização imediata)

Exportando PyTorch para ONNX

ONNX, TensorRT e Deploy Prático

Exemplo resumido de exportação

```
model.eval()  
dummy = torch.randn(1, 3, 224, 224)  
torch.onnx.export(model, dummy, 'modelo.onnx',  
                  input_names=['image'], output_names=['logits'],  
                  dynamic_axes={'image': {0: 'batch'}},  
                  opset_version=17)
```

ONNX Runtime: Inferência Eficiente

ONNX, TensorRT e Deploy Prático

O que é ONNX Runtime (ORT)?

- Motor de inferência de alto desempenho desenvolvido pela Microsoft
- Suporta CPU, GPU NVIDIA (CUDA/TensorRT), GPU AMD (ROCm), Apple ANE, OpenVINO
- Escrito em C++ com bindings para Python, C#, Java, JavaScript, Swift
- Gratuito, open source, amplamente usado em produção (Microsoft, LinkedIn, etc.)

Execution Providers (EP) - backends de aceleração

- CPUExecutionProvider: padrão, funciona em qualquer máquina
- CUDAExecutionProvider: GPU NVIDIA via CUDA
- TensorrtExecutionProvider: delega para TensorRT automaticamente
- OpenVINOExecutionProvider: otimizado para CPU Intel, VPU
- CoreMLExecutionProvider: Apple Silicon e iOS

ONNX Runtime: Inferência Eficiente

ONNX, TensorRT e Deploy Prático

Selecionando EP na pratica

```
import onnxruntime as ort
providers = ['CUDAExecutionProvider',
            'CPUExecutionProvider']
sess = ort.InferenceSession('modelo.onnx',
                             providers=providers)
# ORT usa o primeiro EP disponível na lista
```

Quantizacao: FP32, FP16, INT8

ONNX, TensorRT e Deploy Prático

O que é quantização e por que usar?

- Reduzir a precisão numérica dos pesos e ativações do modelo
- FP32 (32 bits) → FP16 (16 bits) → INT8 (8 bits) → INT4 (4 bits)
- Ganhos: menor tamanho em disco, menor uso de memória, maior throughput

Quantizacao: FP32, FP16, INT8

ONNX, TensorRT e Deploy Prático

FP16 - Half precision

- Redução de 50% no tamanho dos pesos em relação ao FP32
- GPUs modernas (Ampere+) tem unidades Tensor Cores para FP16 nativo
- **Queda de acurácia geralmente < 0.1% em modelos de visão!!!**
- Suportado no ONNX Runtime com CUDAXecutionProvider e TensorRT

INT8 - Quantização inteira

- Redução de 75% no tamanho | velocidade 2-4x maior em hardware compatível
- Requer calibração: coletar estatísticas de ativações em dados representativos
- Post-Training Quantization (PTQ): simples, sem retreinamento
- Quantization-Aware Training (QAT): melhor acurácia, requer retreinamento
- Impacto em modelos de detecção: queda de 0.5-2% em mAP tipicamente

Exemplo Pratico: Modelo timm em FP16

ONNX, TensorRT e Deploy Prático

timm - PyTorch Image Models

- Biblioteca com centenas de modelos de visão pre-treinados (rwightman/timm)
- Modelos: ResNet, EfficientNet, ViT, ConvNeXt, DeiT, Swin, MaxViT, etc.
- Ideal para transfer learning e benchmarking de backbones

Exemplo Prático: Modelo timm em FP16

ONNX, TensorRT e Deploy Prático

Pipeline: timm FP32 → ONNX → ONNX Runtime FP16

```
import timm, torch
model = timm.create_model('efficientnet_b0', pretrained=True).eval()
# Converter modelo para FP16
model_fp16 = model.half().eval()
dummy = torch.randn(1, 3, 224, 224, dtype=torch.float16)
torch.onnx.export(model_fp16, dummy, 'effnet_fp16.onnx',
                  input_names=['image'], output_names=['logits'], opset_version=17)
```

Inferência com ONNX Runtime usando FP16

```
sess_options = ort.SessionOptions()
sess = ort.InferenceSession('effnet_fp16.onnx',
                           providers=['CUDAExecutionProvider'])
result = sess.run(None, {'image': img_fp16})
# Redução típica de ~50% no tempo de inferência vs FP32 em GPU
```

O que é TensorRT?

ONNX, TensorRT e Deploy Prático

NVIDIA TensorRT - SDK de otimização para inferência

- SDK proprietário da NVIDIA para otimizar e acelerar inferência em GPUs NVIDIA
- Produz engines binários altamente otimizados especificamente para o hardware alvo
- Ganhos típicos: 2x a 10x em relação ao PyTorch padrão (FP32)
- Suporta FP32, FP16 e INT8 com calibração

O que é TensorRT?

ONNX, TensorRT e Deploy Prático

O que o TensorRT faz internamente?

- Layer fusion: combina múltiplas operações em um único kernel CUDA
- Kernel auto-tuning: seleciona o kernel CUDA mais rápido para o hardware
- Memory optimization: reutiliza buffers e minimiza transferências GPU-CPU
- Precision calibration: calibra limites para INT8 mantendo acurácia
- Dynamic tensor memory: aloca e libera memória de forma eficiente

Limitações importantes a conhecer

- Engine gerado é específico para o GPU model e TensorRT version (**nao portátil!**)
- Tempo de build do engine pode ser longo (minutos) - cache o engine gerado
- Alguns operadores PyTorch não tem suporte nativo (exigem plugins customizados)
- Requer NVIDIA GPU - **sem suporte a AMD, Intel ou ARM CPU**

Pipeline de Otimização com TensorRT

ONNX, TensorRT e Deploy Prático

Dois caminhos para criar um engine TensorRT

Caminho 1: ONNX → TensorRT (recomendado)

```
import tensorrt as trt
logger = trt.Logger(trt.Logger.WARNING)
builder = trt.Builder(logger)
network = builder.create_network(1 <<
trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
parser = trt.OnnxParser(network, logger)
parser.parse_from_file('modelo.onnx')
config = builder.create_builder_config()
config.set_flag(trt.BuilderFlag.FP16) # habilitar FP16
engine = builder.build_serialized_network(network, config)
```

Pipeline de Otimização com TensorRT

ONNX, TensorRT e Deploy Prático

Dois caminhos para criar um engine TensorRT

Caminho 2: torch2trt / torch-tensorrt (direto do PyTorch)

- torch-tensorrt: compila modelo PyTorch diretamente para TensorRT
- Mais simples de usar, mas menor controle sobre o grafo:

```
import torch_tensorrt
trt_model = torch_tensorrt.compile(model, inputs=[...])
```

TensorRT INT8: Calibração e Precisão

ONNX, TensorRT e Deploy Prático

Por que a calibração é necessária para INT8?

- Pesos podem ser quantizados estaticamente (min/max por camada)
- Ativações variam com os dados - é preciso coletar estatísticas reais
- Calibração: rodar forward pass em ~500-1000 amostras representativas
- Resultado: tabela de escala por tensor usada para mapear FP32 → INT8

Algoritmos de calibração no TensorRT

- IInt8EntropyCalibrator2: padrão e recomendado para a maioria dos modelos
- IInt8MinMaxCalibrator: usa min/max absoluto - mais conservador
- IInt8PercentileCalibrator: percentil configurável - útil para outliers
- IInt8LegacyCalibrator: compatibilidade retroativa

TensorRT INT8: Calibração e Precisão

ONNX, TensorRT e Deploy Prático

Dica prática: cache de calibração

```
config.int8_calibrator = MyCalibrator(calib_data)
config.set_flag(trt.BuilderFlag.INT8)
# Salvar cache: evita recalibrar a cada build
calibrator.set_cache('calibration.cache')
```

- Use INT8 quando a GPU não tem Tensor Cores (ex: Jetson Nano, GTX série)

Comparativo de Performance

ONNX, TensorRT e Deploy Prático

Benchmarks típicos - ResNet50, batch=1, GPU RTX 3060

- PyTorch FP32: ~5-8 ms por imagem (baseline)
- ONNX Runtime CPU (FP32): ~30-60 ms por imagem
- ONNX Runtime GPU (FP32): ~4-6 ms por imagem
- ONNX Runtime GPU (FP16): ~2-4 ms por imagem
- TensorRT FP32: ~3-5 ms por imagem
- TensorRT FP16: ~1.5-3 ms por imagem
- TensorRT INT8: ~1-2 ms por imagem

Comparativo de Performance

ONNX, TensorRT e Deploy Prático

ONNX Runtime CPU - quando faz sentido

- Maquinas sem GPU: laptop corporativo, servidor CPU-only
- Modelos leves: MobileNetV3, EfficientNet-B0, modelos TIMM compactos
- Paralelismo: multiplos processos em servidor multi-core
- Ganho tipico do ORT vs PyTorch em CPU: 1.5x - 3x

Conclusão prática

- Para CPU pura: ONNX Runtime é a melhor opção (portabilidade + desempenho)
- Para GPU NVIDIA: TensorRT FP16 é melhor opção de performance
- Para GPU pequena (sem Tensor Cores FP16): INT8 com calibração

Deploy sem GPU: Alternativas ao TensorRT

ONNX, TensorRT e Deploy Prático

Quando o TensorRT não é uma opção

- GPU AMD, Intel integrada, ARM CPU, Apple Silicon, FPGA
- Máquinas de produção sem GPU (servidores de inferência CPU-only)
- Dispositivos IoT de baixo custo

YOLOv8n: Arquitetura e Características

Deploy Edge

A família YOLOv8 da Ultralytics

- Lancado em 2023, sucessor do YOLOv5 - arquitetura redesenhada
- Cinco variações: n (nano), s (small), m (medium), l (large), x (xlarge)
- Tarefas: detecção, segmentação, classificação, pose estimation, OBB
- API unificada em Python e CLI para treino, validação, predição e exportação

YOLOv8n: Arquitetura e Características

Deploy Edge

YOLOv8n - o modelo nano

- Apenas 3.2M de parametros e 8.7 GFLOPs
- mAP50-95 em COCO: ~37.3 (vs 53.9 do YOLOv8x)
- Velocidade em CPU (ONNX): ~80-100ms por frame | GPU: ~1-3ms
- Tamanho do modelo: ~6 MB (FP32) → ~3 MB (FP16) → ~1.5 MB (INT8)

Por que YOLOv8n para drone?

- Cabe na memória limitada de hardware embarcado (< 4 GB RAM típico)
- Velocidade suficiente para 15 FPS em hardware de borda com otimização
- Ultralytics exporta diretamente para TensorRT, ONNX, ncnn sem código adicional
- Treinamento incremental: fine-tuning com poucas centenas de imagens do drone

Drones: Desafios de Deploy

Deploy Edge

Restricoes criticas de um drone

- Consumo de energia: cada watt adicional reduz tempo de voo
- Peso: computador de bordo deve ser o mais leve possível (< 200g tipico)
- Vibrações: afetam camera e can introduzir ruido nas imagens
- Temperatura: dissipação de calor em espaco confinado sem ventilação
- Latência máxima: detecção deve ocorrer em < 100ms para controle em loop fechado

Drones: Desafios de Deploy

Deploy Edge

Desafios da visão computacional em drones

- Perspectiva aérea: objetos pequenos, oclusão parcial, distorção
- Variação de altitude: escala dos objetos muda continuamente
- Motion blur: movimento rápido causa desfoque nas imagens capturadas
- Iluminação variável: sombras, reflexos, contraluz em missões externas
- Dataset shift: modelo treinado em imagens ground-level pode falhar com imagens aéreas

Pipeline de Exportação YOLOv8n

Deploy Edge

Ultralytics oferece exportação direta em uma linha de comando

- Suporta 12+ formatos: ONNX, TensorRT, CoreML, TFLite, ncnn, OpenVINO, etc.
- Inferência e validação pos-exportação integrada ao CLI

Exportando para ONNX (formato base portavel)

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
model.export(format='onnx', imgsiz=640, half=True,
              dynamic=False, simplify=True, opset=17)
```

- half=True: exporta FP16 | simplify=True: aplica onnx-simplifier automaticamente

Pipeline de Exportação YOLOv8n

Deploy Edge

Exportando diretamente para TensorRT (em maquina com GPU NVIDIA)

```
model.export(format='engine', imgsz=640, half=True,  
            device=0, workspace=4)
```

- format='engine': gera .engine TensorRT | workspace: GB de workspace
- half=True: FP16 nativo no TensorRT
- Resultado: arquivo .engine especifico para o GPU do Jetson - não portavel!

Escolhendo o Formato Certo para o Drone

Deploy Edge

Decisão por hardware

Jetson Orin NX ou Jetson Xavier (GPU Ampere/Volta)

- Melhor opção: TensorRT FP16 (.engine)
- Por que: GPU dedicada com Tensor Cores, suporte nativo TensorRT
- Ganho esperado vs PyTorch: 5-10x em throughput
- Cuidado: gerar o .engine no próprio Jetson alvo (não é portável entre GPUs!)

Raspberry Pi 5 + Hailo-8 (NPU externo)

- Melhor opção: HailoRT com modelo no formato .hef (Hailo Execution Format)
- Alternativa: ONNX Runtime com OnnxruntimeExecutionProvider para CPU
- ncnn como fallback se não tiver NPU

Escolhendo o Formato Certo para o Drone

Deploy Edge

Decisão por hardware

Drone sem GPU dedicada (CPU ARM pura)

- Melhor opção: ncnn (YOLOv8n ncnn) ou TFLite INT8
- Resolução menor (320x320 ou 416x416) para atingir FPS aceitável
- Usar modelos ainda menores: YOLOv8n-world ou YOLO-NAS-s

Jetson Nano (GPU Maxwell - sem Tensor Cores FP16)

- Melhor opção: TensorRT INT8 com calibração
- FP16 não tem aceleração nesta geração - INT8 é mais rápido
- Esperar 8-15 FPS com YOLOv8n INT8 em 640x640

Validação e Metricas Apos Exportacao

Deploy Edge

Por que validar o modelo exportado?

- Erros de quantização podem degradar acurácia de forma não uniforme
- Operadores não suportados podem ser ignorados silenciosamente
- O modelo ONNX/TensorRT pode ter comportamento diferente do PyTorch original

Validação com Ultralytics CLI

```
model_onnx = YOLO('yolov8n.onnx')  
metrics = model_onnx.val(data='coco8.yaml', imgsz=640)  
print(metrics.box.map)    # mAP50-95
```

- Comparar com baseline PyTorch: tolerancia de < 1-2% de queda em mAP

Validação e Metricas Apos Exportacao

Deploy Edge

Benchmarking de latência

```
model.benchmark(imgsz=640, half=True, device=0)
```

- Mede latência media em ms e FPS para cada formato exportado
- Executar no hardware ALVO (nao no servidor de treinamento!)

Metricas de produção a monitorar

- Latência P50, P95, P99 (percentis) - não apenas a media
- Throughput (FPS) com e sem pre/pos-processamento
- Consumo de memoria GPU/CPU em pico
- Taxa de falsos positivos e falsos negativos por categoria de objeto

Dicas Praticas para Deploy em Drones

Deploy Edge

Pre-processamento otimizado

- Usar resolução 320x320 ou 416x416 ao inves de 640x640 - ganho de 2-3x em FPS
- Pre-processar imagens diretamente em GPU (torchvision ou CUDA kernels)
- Usar letterbox padding consistente com o treinamento
- Normalizar na GPU: evitar round-trip CPU-GPU por frame

Fine-tuning para imagens aereas

- Coletar dataset de imagens aereas com labels especificos para o caso de uso
- Transfer learning: partir do yolov8n.pt pre-treinado em COCO
- Aumentação de dados: rotação 360 graus, simulação de altitude variavel
- Apos fine-tuning: reexportar e recalibrar o engine TensorRT

